

Advanced Topics in Automation and Control Engineering

Aerial Robotics Laboratory

Trajectory generation

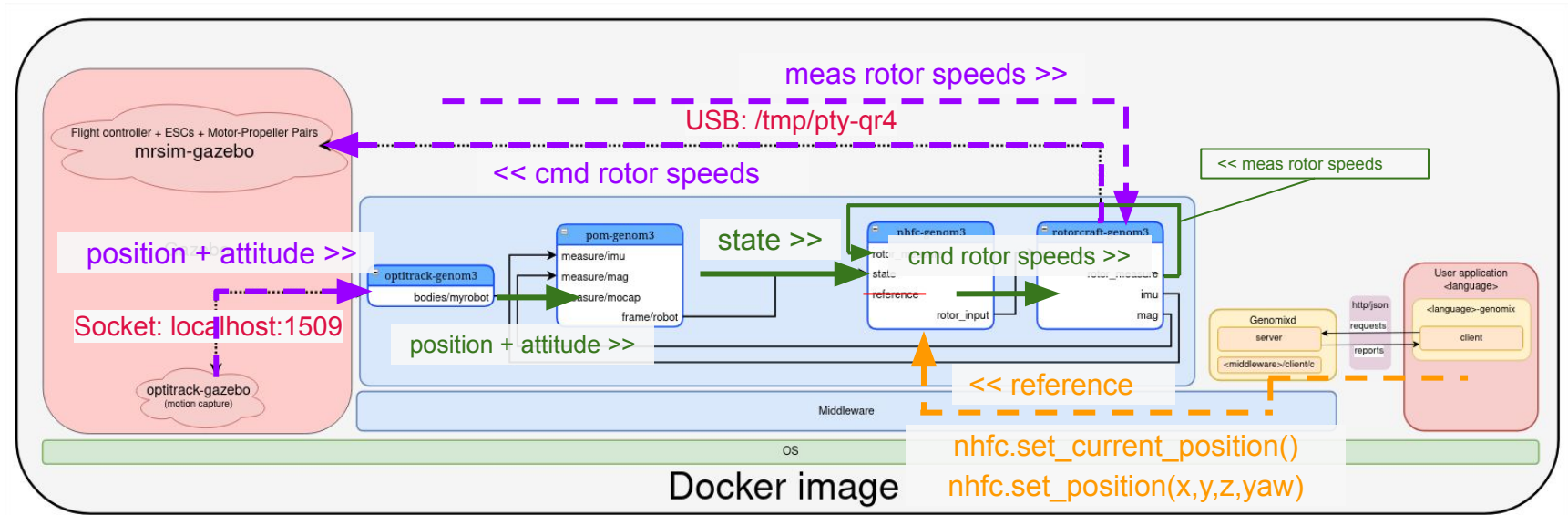
Dr Marco Tognon
Dr Gianluca Corsini



A quick recap first ...

telekyb3 – Quadrotor simulation in Gazebo

Visual representation of the data flow



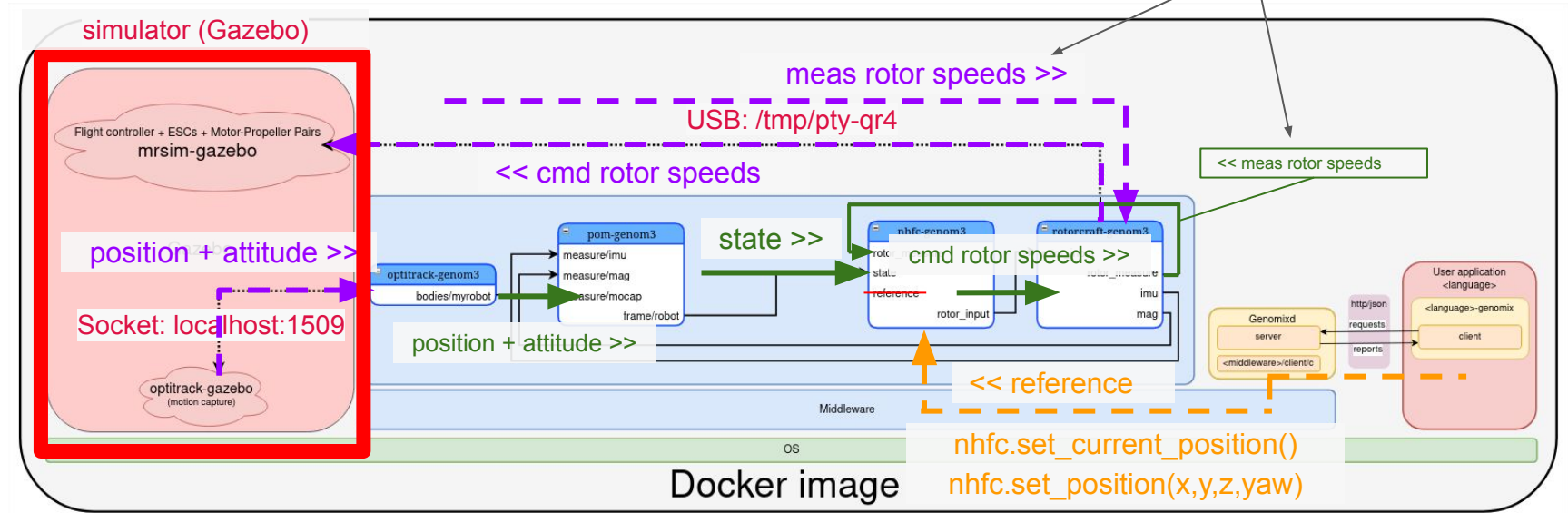
where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))

- *state* := position, attitude, linear and angular velocities, linear and angular accelerations

telekyb3 – Quadrotor simulation in Gazebo

Visual representation of the data flow



where

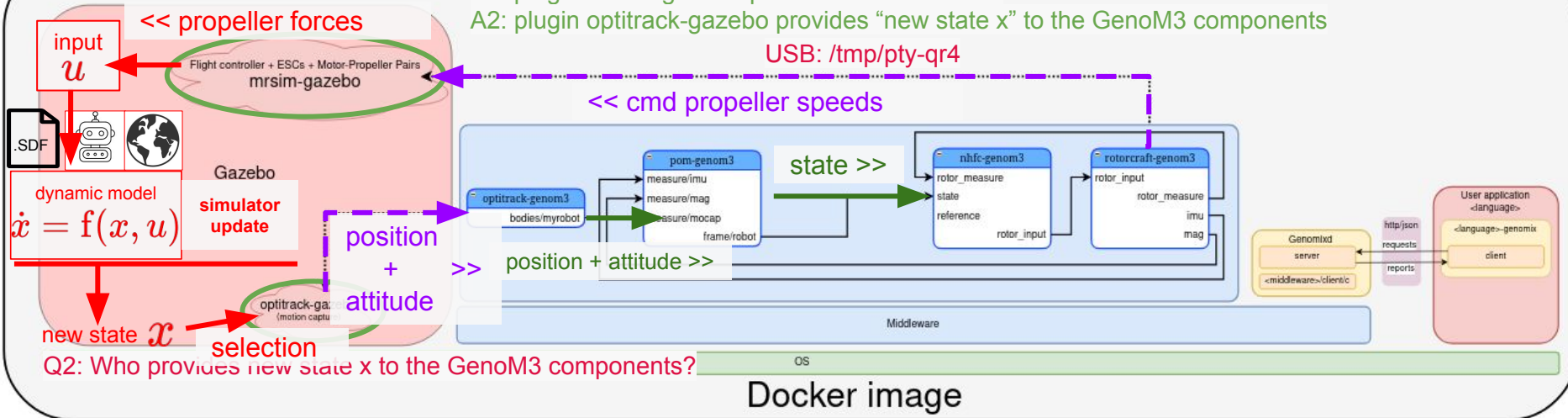
- `<middleware>` = [pocoLibs](#)
- `<language>` = python ([python-genomix documentation](#))

- `state` := position, attitude, linear and angular velocities, linear and angular accelerations

telekyb3 – Quadrotor simulation in Gazebo

What happens inside Gazebo?

Q1: Who provides input u to simulator?



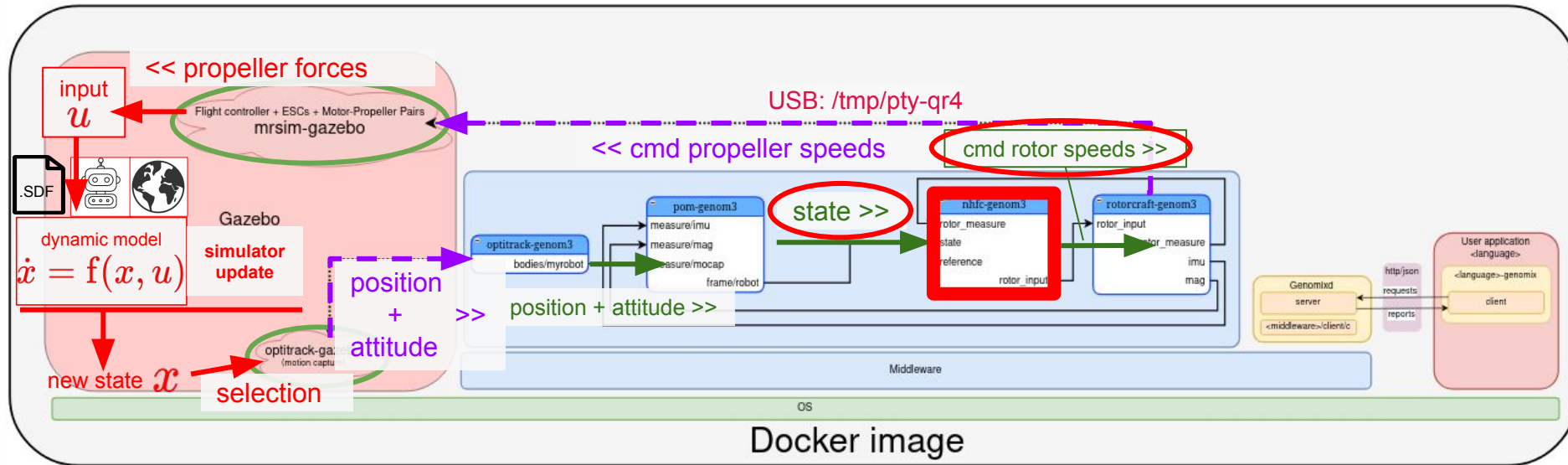
where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))

telekyb3 – Quadrotor simulation in Gazebo

What if we want to replace Gazebo?

Let's consider I/O of the controller (nhfc)



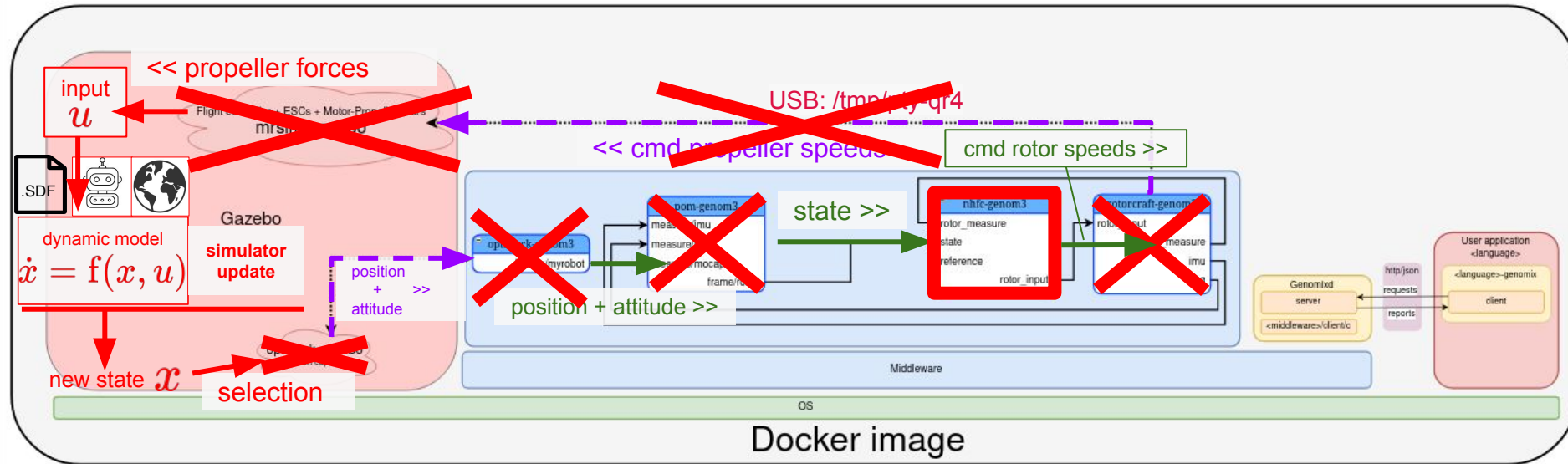
where

- **<middleware>** = [pocoLibs](#)
- **<language>** = *python* ([python-genomix documentation](#))

telekyb3 – Quadrotor simulation in Gazebo

What if we want to replace Gazebo?

Assumption: no hardware-in-the-loop simulation!



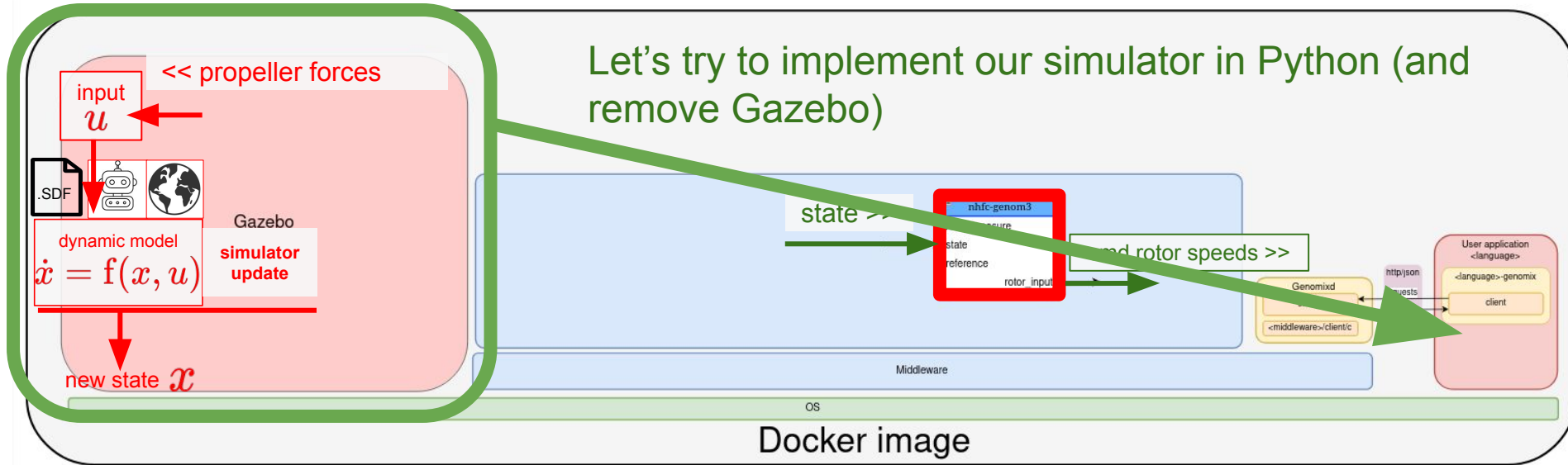
where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))

telekyb3 – Quadrotor simulation in Gazebo

What if we want to replace Gazebo?

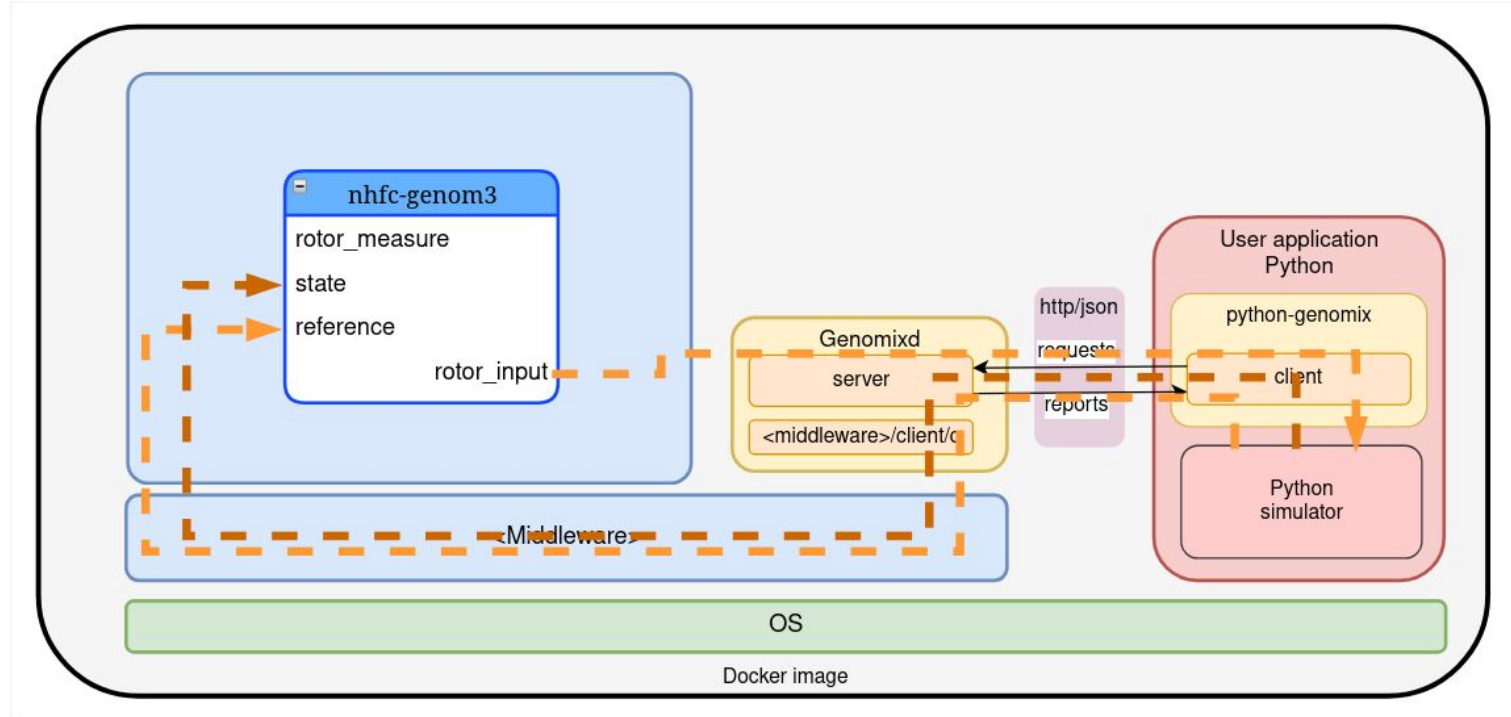
Assumption: no hardware-in-the-loop simulation!



where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-geomix documentation](#))

telekyb3 – Quadrotor simulation in Python



What you learnt so far about telekyb3 and Gazebo

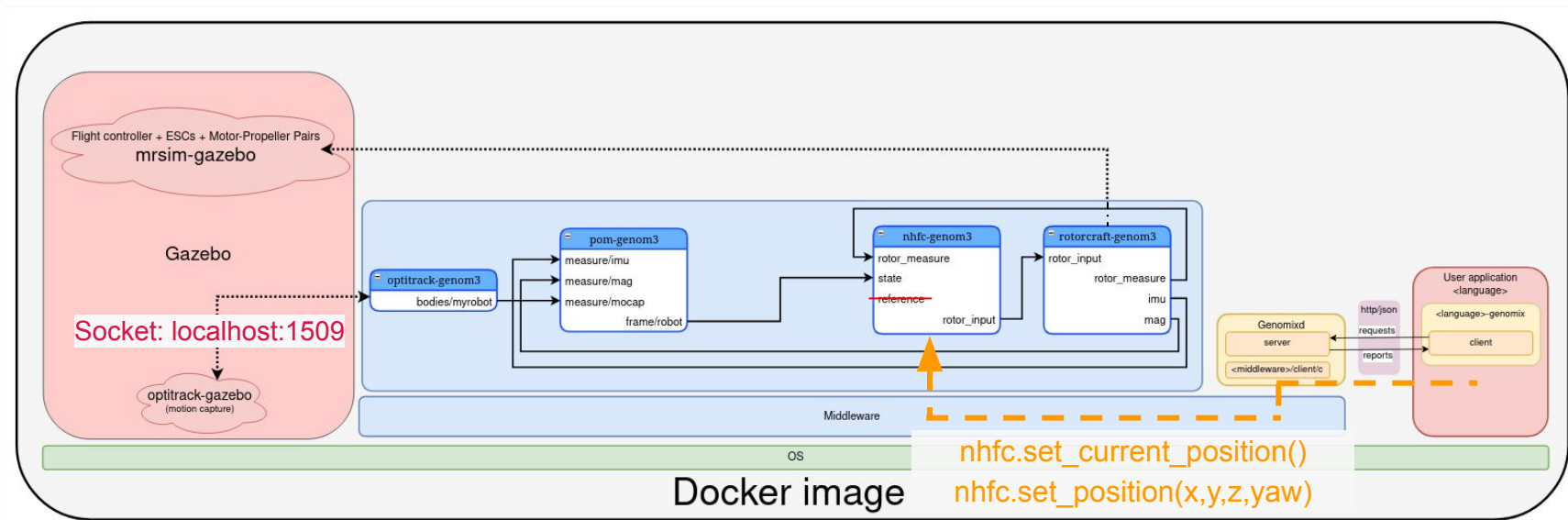
1. How to change change robot and world conditions in the Gazebo simulator
 - a. SDF specifications
 - b. What the telekyb3 Gazebo plugins do
 - c. Working principles of the Gazebo simulation engine
2. How to run a middleware (pocoLibs) and the necessary GenoM3 components
3. How to interact with the GenoM3 components from a Python user application (python-genomix)
 - a. Call services (e.g., connect ports, start, stop, servo, set/get values, ...)
 - b. Read data from ports
 - c. Write data to ports
4. How to build data types exchanged by GenoM3 components through ports in Python
 - a. openrobots-idl
 - b. python-genomix mappings



Today...

How to generate the controller reference?

Up to now, the controller receives “**constant**” **set-points** through services called thanks to python-genomix.

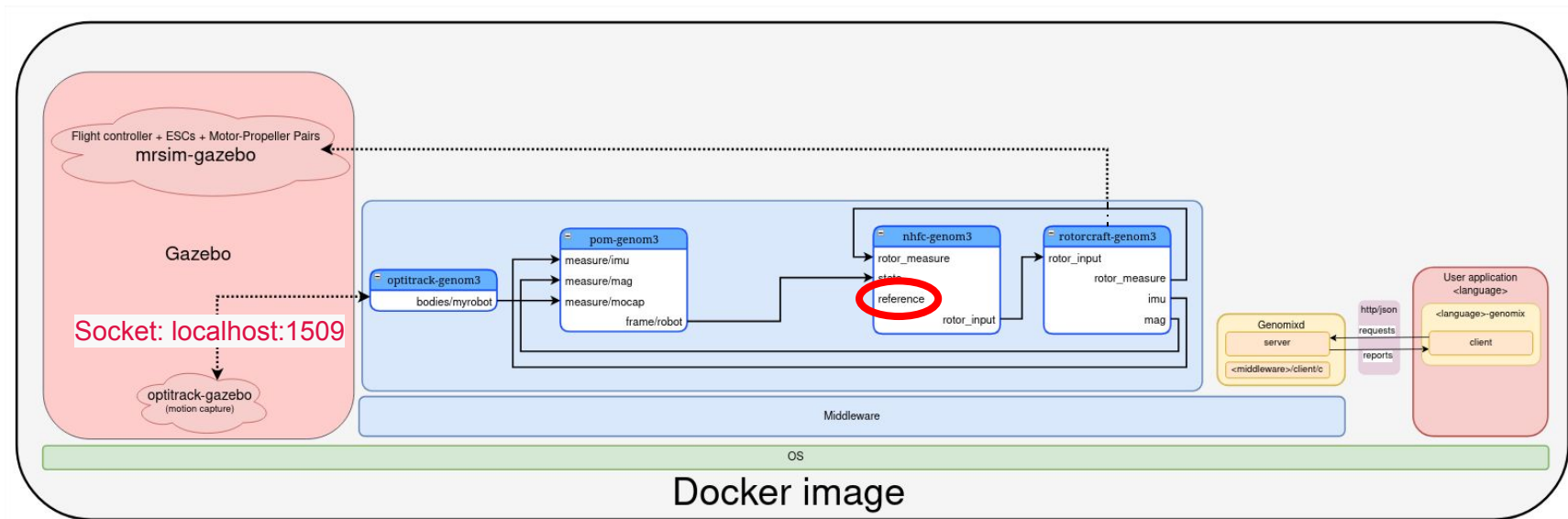


where

- **<middleware>** = [pocoLibs](#)
- **<language>** = *python* ([python-genomix documentation](#))

How to generate the controller reference?

What about nhfc's input port **reference**?



where

- `<middleware>` = [pocoLibs](#)
- `<language>` = `python` ([python-genomix documentation](#))

How to generate the reference?

As for any extension of the telekyb3 framework, you can choose among

1. developing a **GenoM3 component**
2. or writing an **user application** (e.g., in Python and with python-genomix)

that produces the *reference* trajectory to the controller (in our case, nhfc-genom3).

1. GenoM3 component for reference trajectory

An user can develop a GenoM3 component producing a reference trajectory starting from the current robot state.

In telekyb3, there is already a GenoM3 component developed for generating trajectories:

[maneuver-genom3](#)

This component produces a reference trajectory starting from the current robot state.

In particular, maneuver-genom3 implements a “[...] *Kinodynamic Trajectory Planner*: this component is a spline-based, minimum-time, constant-snap trajectory generator. It applies to any robot with decoupled dynamics or differentially flat.”

maneuver-genom3

Ports:

- [state \(in\)](#): when generating trajectories, it uses the current robot state as initial state which is read from this input port
- [desired \(out\)](#): output port containing the planned trajectory

Services:

- [set_bounds \(function\)](#): Configure the limits of the free space. The **x**, **y** and **z** minimum and maximum bounds limit the extend of the planned motion. Any trajectory going beyond these limits will be silently clipped. **yaw** limits in the same way the heading of the robot.
- [set_current_state \(activity\)](#): Set initial planning position to current one
- [take_off \(activity\)](#): Vertical take-off or landing from current state. **take_off** plans and executes a trajectory from the current internal state (see [set_current_state \(activity\)](#)) to the state corresponding to the same X, Y position and the absolute Z coordinate equal to **height**. This position is reached with a 0 acceleration and velocity. Any ongoing motion is cancelled and the planned trajectory is executed immediately.
- [goto \(activity\)](#): Reach a given position from current state.
- [waypoint \(activity\)](#): Push a given position to be reached after the last one.

NB: Above only the relevant ports and services are listed, a full list is provided in the [component documentation](#).

2. User application for reference trajectory

An user can develop an user application (e.g., in Python with python-genomix) producing a reference trajectory starting from the current robot state and feeding it to the controller (nhfc-genom3).

How to generate the reference?

Why not using polynomial trajectories?

- **Smooth** → Differentiable (prevent discontinuities for velocity, acceleration, jerk, ...)
- **Flexible** → Satisfy boundary conditions (start/end position, velocity, ...)
- **Analytical** (closed-form) **solutions** and **easy to compute** (simple algebra)

Polynomial trajectory generation

We describe the trajectory for one joint or coordinate as a polynomial in time:

$$q(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n$$

where:

- t : time
- $q(t)$: position over time
- $a_0, a_1, \dots, a_n$ are the coefficients
- n : polynomial degree

Polynomial trajectory generation

How to compute the coefficients $a_0, a_1, \dots, a_n$?

$$q(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n$$

1. **We set boundary conditions** such as

At initial time $t=0$:

- $q(0) = q_0$
- $\dot{q}(0) = v_0$
- $\ddot{q}(0) = a_0$
- [...]

At final time $t=T$:

- $q(T) = q_f$
- $\dot{q}(T) = v_f$
- $\ddot{q}(T) = a_f$
- [...]

The **order of the polynomial** must be $n = \text{\#boundary_conditions} - 1$

Polynomial trajectory generation

How to compute the coefficients $a_0, a_1, \dots, a_n$?

$$q(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_n t^n$$

2. **Compute all the necessary derivatives, e.g.**

$$\dot{q}(t) = a_1 + 2a_2 t + 3a_3 t^2 + 4a_4 t^3 + 5a_5 t^4$$

$$\ddot{q}(t) = 2a_2 + 6a_3 t + 12a_4 t^2 + 20a_5 t^3$$

[...]

3. **Build a linear system as $A\mathbf{a} = \mathbf{b}$ and solve it for $\mathbf{a}$, where $\mathbf{a} = [a_0 \ a_1 \ a_2 \ \dots \ a_n]^T$**

4. **Interpolate $q(t)$ and its derivatives at time t to get smooth trajectory**

Quintic polynomial trajectory generation

Closed-form solutions to previous problem exist!

For instance, if we consider **quintic (n=5) polynomial trajectories**, we have that

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

$$\dot{q}(t) = a_1 + 2a_2t + 3a_3t^2 + 4a_4t^3 + 5a_5t^4$$

$$\ddot{q}(t) = 2a_2 + 6a_3t + 12a_4t^2 + 20a_5t^3$$

At $t = 0$:

- $q(0) = q_0$
- $\dot{q}(0) = v_0$
- $\ddot{q}(0) = a_0$

At $t = T$:

- $q(T) = q_f$
- $\dot{q}(T) = v_f$
- $\ddot{q}(T) = a_f$

you obtain that

$$a_0 = q_0 \quad a_3 = \frac{10\Delta q}{T^3} - \frac{4\Delta v}{T^2} + \frac{1}{2} \frac{\Delta a}{T}$$

$$a_1 = v_0 \quad a_4 = -\frac{15\Delta q}{T^4} + \frac{7\Delta v}{T^3} - \frac{\Delta a}{T^2}$$

$$a_2 = \frac{a_0}{2} \quad a_5 = \frac{6\Delta q}{T^5} - \frac{3\Delta v}{T^4} + \frac{1}{2} \frac{\Delta a}{T^3}$$

where

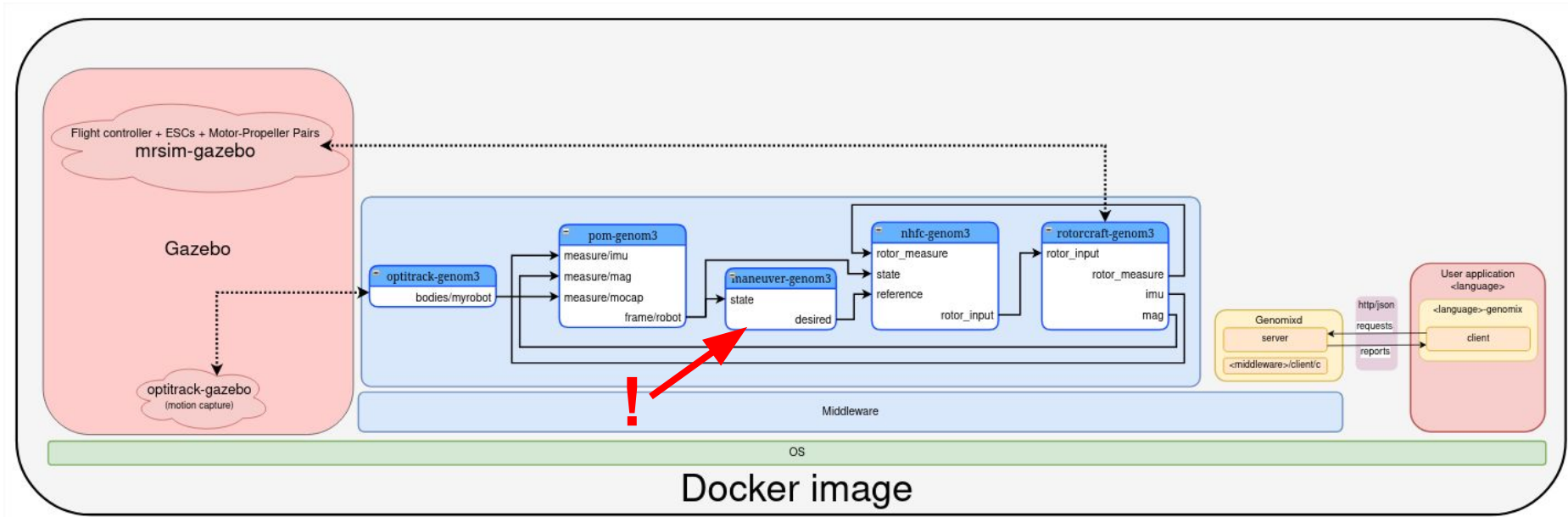
$$\Delta q = q_f - \left(q_0 + v_0T + \frac{1}{2}a_0T^2 \right)$$

$$\Delta v = v_f - (v_0 + a_0T)$$

$$\Delta a = a_f - a_0$$

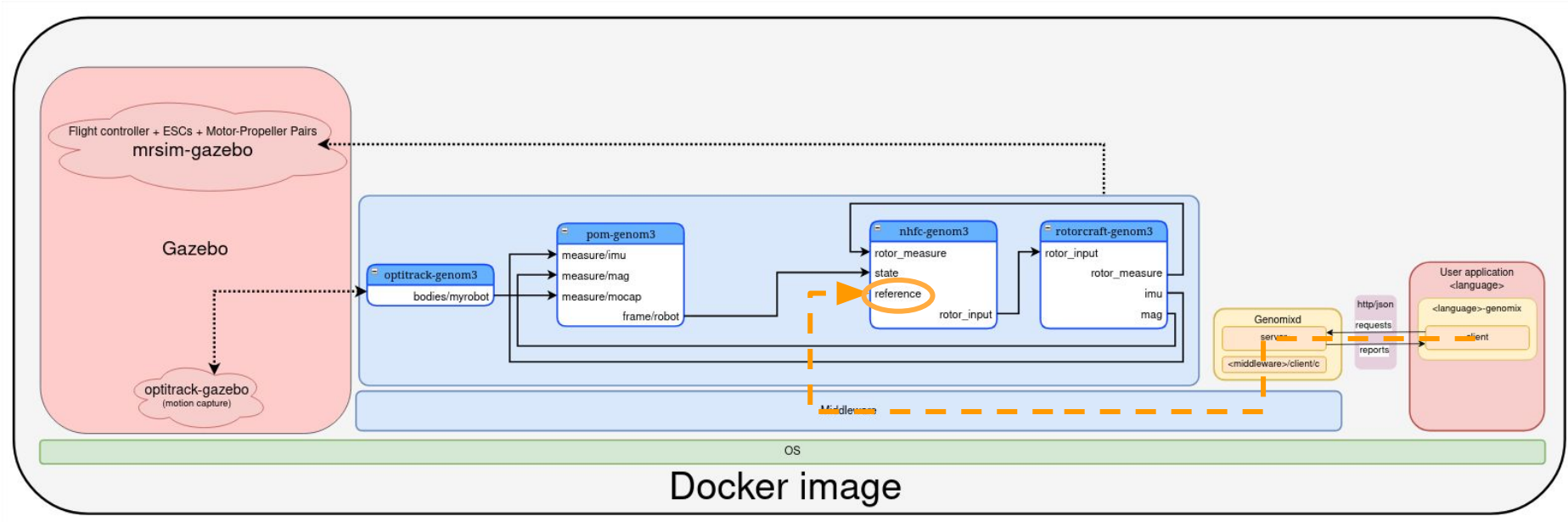
Assignment #1

Starting from the previous simulations, re-make those simulations, but this time use also the component **maneuver-genom3** for providing the reference trajectory to the controller nhfc-genom3.



Assignment #2

Write an user application (Python + python-genomix) that generates a reference trajectory and feeds it to the input port **reference** of the controller nhfc-genom3.



Assignments guidelines

1. Repeat all the Gazebo simulations (quadrotor, hexarotor, ...), this time
 - a. with maneuver-genom3 providing the reference trajectory in x, y, z, and yaw to the controller nhfc-genom3
 - b. with an user application generating the reference trajectory in x,y, z and yaw to the controller nhfc-genom3
2. Use same waypoints as before
3. Use time durations of your choice
4. Realize plots
 - a. when using maneuver-genom3 → new log file to parse for the plots
 - b. when using your user application in Python → generate log file as maneuver-genom3

telekyb3 – Invoking services

Up to now, in the user application, you invoked services *synchronously*.

See [synchronous call documentation](#) for invoking services with python-genomix.

However, services can be invoked also *asynchronously*.

See [asynchronous call documentation](#) for details.

What the user shall know:

1. Services have statuses: **sent**, **acknowledged**, **done**, **error**, ...
2. Service call workflow:
 - a. The client sends a service request to a GenoM3 component → status: **sent**
 - b. The component starts the service and sends an acknowledgment reply → status: **acknowledged**
 - c. The component completes the service and sends a final reply → status: **done**

telekyb3 – Synchronous vs asynchronous

Synchronous call

- The client calls a service request and **waits**
- The call **blocks** until the service completion
- The result is returned **directly**
- Simple to use, but no parallel work possible

“Call → wait → result”

Asynchronous call

- The client sends a service request and **continues execution**
- The call is **non-blocking**
- The client tracks the service via **statuses** (**sent** → **acknowledged** → **done / error**) and can wait for a status to be reached
- The result can be obtained later
- Parallel work is now possible

“Send [→ Wait for ack] [→ monitor] [→ retrieve result]”

Key difference

- **Synchronous**: blocking, immediate result
- **Asynchronous**: non-blocking, result via status updates

telekyb3 – Invoking services asynchronously

You can invoke a service asynchronously by adding one of the two following optional arguments to a service call:

- **send=True** → **just send the request, do not wait for any acknowledgement**
 - The request is sent and the service method returns immediately
 - No waiting for acknowledgment or result
 - Useful when the acknowledgement is not needed
- **ack=True** → **send the request, wait for acknowledgement and return**
 - The request is sent and the service method returns after acknowledgment
 - Useful to know that the service has started, but no need to wait for its completion

In both cases:

- The client receives a **service handle** (same as for the synchronous invocation)
- The service status can be monitored: **sent** → **acknowledged** → **done / error**
- The result can be retrieved later

telekyb3 – Example of invoking services asynchronously

SYNCHRONOUS

```
>>> maneuver.goto(x=1, y=1, z=1, yaw=0, duration=5)
```

The client sends the request and then it waits for the service to end. Thus the client will wait for 5s as maneuver will produce a reference for 5s and then it will stop updating its output port.

ASYNCHRONOUS

```
>>> req1 = maneuver.goto(x=1, y=1, z=1, yaw=0, duration=5, send=True)
```

Just send the request, do not wait for an acknowledgement. Here, the client sends the request and it waits neither for its completion nor for an acknowledgement. The code will not stop and proceed with the next instructions!

```
>>> req2 = maneuver.goto(x=1, y=1, z=1, yaw=0, duration=5, ack=True)
```

Send the request, but wait for an acknowledgement. Here, the client sends the request and it waits for an acknowledgement. The code will not stop and proceed with the next instructions!

telekyb3 – Example of invoking services asynchronously

At this point, the services invoked asynchronously could be still running within the component. Thus, you may want to check their statuses or wait for their completions.

print request statuses

```
>>> print(req1.status)
```

```
>>> sent
```

```
>>> print(req2.status)
```

```
>>> sent
```

wait for request completions

```
>>> req1.wait()
```

```
>>> req2.wait()
```

print request statuses

```
>>> print(req1.status)
```

```
>>> done
```

```
>>> print(req2.status)
```

```
>>> done
```

get the request results

```
>>> res1 = req1.result()
```

```
>>> res2 = req2.result()
```

telekyb3 – Performance in invoking services

For every service invocation, genomix internally tracks the status, which requires server-client communication.

This could jeopardize performance: network delays, more computational load (more instructions to run), memory (store all service requests, statutes, ...), ...

Solution?

`oneway=True` → **minimal communication**

- The request is sent **without any reply expected**
- No acknowledgment, no final response
- The call returns **immediately (non-blocking)**
- No service handle, no status tracking (no status monitoring and waiting capabilities)

telekyb3 – oneway=True

Why is it faster?

- Avoids **network round-trips** (no ack, no result message)
- Reduces **communication overhead**
- Lower latency → better for **high-frequency calls**

When to use it

- The result is **not needed**
- The service is **short / frequent**
- You want **maximum performance**

Trade-off

- No guarantee the service started, completed, succeeded

telekyb3 – Example oneway=True

```
# ask rotorcraft 70Hz as commanded frequency for all 8 propellers  
# fire and forget  
>>> rotorcraft.set_velocity([70 for i in range(8)], oneway=True)  
# NB: this will come handy in the next class
```